De SOLID ontwerpprincipes

In dit document worden kort een aantal basisprincipes van objectgeoriënteerd werken herhaald.

Daarna worden de SOLID-principes voorgesteld. Die zijn een poging om een aantal vuistregels voor goed objectgeriënteerd ontwerp samen te brengen in een letterwoord dat in het geheugen blijft hangen.

De SOLID-principes zijn niet de enige ontwerpprincipes die er bestaan. Je zult er in de loop van je opleiding en je carrière zeker nog andere ontmoeten.

1 Objecten, objectgeoriënteerd werken

Wat in dit hoofdstuk staat zou een herhaling moeten zijn van dingen die je al eens gelezen of gehoord hebt.

1.1 Wat is een object?

Een object is een stuk software, een samenhangend geheel van data en methodes.

1. Een object heeft een **identiteit**.

   Dat wil zeggen: elk object is uniek. In die zin zijn objecten net als mensen: we lijken allemaal op elkaar, maar elk mens heeft een eigen identiteit. Ik ben iemand, jij bent iemand anders, en dat verschil is voor iedereen duidelijk.

2. Een object heeft een **toestand.**

   Een object kan verschillende toestanden hebben. Die toestand wordt bijgehouden in de *attributen* van het object.

3. Een object heeft een **gedrag**.

   Een object kan dingen doen. Het gedrag wordt bepaald door de *methodes* van het object.

Objecten met gelijkaardige toestanden en gelijkaardig gedrag worden samengebracht in een *klasse*. Elk object is een *instantie* van 1 klasse, en een object kan nooit van klasse veranderen.

Van een object kun je enkel de toestand wijzigen tijdens de uitvoering van een programma. De structuur van het object (welke attributen en methodes het heeft) ligt vast. Het gedrag ligt ook vast. Om de structuur of het gedrag te wijzigen moet je het programma opnieuw compileren.

1.2 Het doel van object-oriëntering

Object-oriëntering is ontstaan uit een nood aan *onderhoudsvriendelijker* en *gemakkelijker te hergebruiken* software.

De onderhoudsvriendelijkheid van software is enorm belangrijk. De meeste softwaresystemen worden tijdens hun levensduur voortdurend aangepast. Er zijn meer informatici bezig met het onderhoud van bestaande systemen dan met het ontwikkelen van nieuwe. Onderhoudsvriendelijke software zorgt er voor dat dit efficiënt kan gebeuren.
Objecten moeten software overzichtelijker en gemakkelijker te begrijpen maken. In software die overzichtelijk is kun je snel terugvinden welke stukken aangepast moeten worden, en welke niet. Daardoor wordt onderhoud gemakkelijker.

Bij het ontwikkelen en onderhouden van software kun je ook veel tijd winnen als je bestaande code opnieuw kunt gebruiken. We vinden het bijvoorbeeld vanzelfsprekend om niet zelf een string-klasse te schrijven, maar om er een te gebruiken die al bestaat.
Objecten zijn bedacht met de bedoeling hergebruik gemakkelijker te maken: een object is een soort pakketje met daarin alles wat nodig is voor een bepaalde functionaliteit.

Zoals veel nieuwe dingen werden ook objecten in het begin heel erg gehypet. Met objecten zou programmeren kinderspel worden. Na een tijd kwam er ontnuchtering, maar het idee was zo goed dat het nu, bijna vijftig jaar later, onbetwist de meest gebruikte manier van programmeren is.

Tegenwoordig is er een trend naar hergebruik op een grotere schaal dan objecten, namelijk met frameworks en pakketten (bijvoorbeeld: een framework voor een webtoepassing, waar je via een package manager bijkomende functionaliteiten kunt aan toevoegen).

Ook functioneel programmeren is erg in opkomst. Objectgeoriënteerde programmeertalen worden vaak uitgebreid met functionele programmeermogelijkheden (bijvoorbeeld: lambda's in Java).
Het klassieke objectgeoriënteerde denken maakt een onderscheid tussen functies en gegevens. Dit onderscheid komt van het klassieke computermodel: er is een processor en een geheugen. De processor voert machine-instructies uit, en die instructies worden beschreven door de functies. Om van de functies machine-instructies te maken heb je een compiler nodig. Maar compilers zijn trage dingen. Toen objectgeoriënteerde talen bedacht werden was compileren enkel praktisch haalbaar als het vooraf gebeurde, voor dat de gebruiker de software krijgt. Gebruikers kregen een gecompileerde versie van het programma, want compileren tijdens het gebruik van het programma was niet haalbaar. Vandaar het onderscheid tussen enerzijds de data, dus de attributen van objecten, waarvan de inhoud gemakkelijk te wijzigen is, en de rest van het object, dus de structuur en de methodes, die niet wijzigbaar zijn.
Computers zijn sindsdien ongelooflijk veel sneller geworden, en men heeft ontdekt dat er een stijl van functies is (functies zonder neveneffecten) die zich goed leent tot parallelliseren. Daardoor is er nu meer mogelijk, en worden functies meer en meer zoals gegevens: iets wat je kunt wijzigen at runtime.
Veel van de patterns die in het handboek aan bod komen hebben hun oorsprong in problemen die ontstaan uit die beperking van objecten: namelijk dat hun structuur en hun methodes niet wijzigbaar zijn. Als die beperking er niet meer is kun je tot heel andere oplossingen komen. Dit valt echter buiten het bestek van dit vak.

1.3 Basisideeën van object-oriëntering

1.3.1 Modulariteit

Ieder object is als het ware een bouwsteen. Een stuk software wordt opgebouwd uit die bouwstenen, in plaats van als één massief geheel.

Goed passende bouwstenen geven een stuk software een *overzichtelijke structuur*.
Goede bouwstenen zijn *uitwisselbaar*.
Die twee zaken maken software gemakkelijker te begrijpen en te onderhouden.

Het is echter niet vanzelfsprekend een goede bouwsteen te maken! De richtlijnen in het vervolg van dit hoofdstuk proberen daarbij te helpen.

1.3.2 Encapsulering

Een goed object heeft een zichtbaar (public) deel en een verborgen (private) deel. Dat is een principe dat je niet enkel in software ziet. Een auto bijvoorbeeld heeft veel onderdelen die verborgen zijn: de motor, de remmen, de stuurinrichting … Die zijn verborgen, omdat het zeer onpraktisch en gevaarlijk zou zijn ze rechtstreeks te bedienen. In de plaats daarvan bestuur je een auto met het stuur en de pedalen (het zichtbare deel). Bij objecten is het net zo: dingen die onpraktisch of gevaarlijk zijn om rechtstreeks te bedienen worden verborgen. Dat is *encapsulering*.

Ook encapsulering is goed voor onderhoudbaarheid. Het private deel van een object is onzichtbaar voor de rest van het programma. Je kunt dat private deel dus wijzigen zonder dat je je zorgen moet maken over de rest van het programma. Je moet enkel zorgen dat het publieke deel van het object blijft werken zoals voordien.

Het zichtbare deel van een object wordt ook wel de *interface* van het object genoemd (niet te verwarren met een interface in Java of C#, dat is iets anders).

1.3.3 Overerving

Het is mogelijk een object te nemen, en daar een nieuw object van af te leiden.

Dat wil zeggen dat het afgeleide object de functionaliteit van het andere object overneemt, of uitbreidt, of specialiseert.

Overerving is een van de manieren waarop je een object kunt hergebruiken.

1.3.4 Polymorfisme

Objecten met een gelijkaardige interface kunnen elkaars plaats innemen. Sterker nog: een object dat een functionaliteit nodig heeft van een ander object hoeft niet te weten welk concreet object dat is, zolang de functionaliteit er maar is. Dat betekent dat we als programmeur niet vooraf moeten

zeggen welke concrete objecten welke taken zullen uitvoeren (‘early binding’). We kunnen die keuze uitstellen tot het allerlaatste moment, nl. de uitvoering van het programma (‘late binding’).

Ook dit is een principe dat in het dagelijks leven veel voorkomt. Als je een klaslokaal binnenkomt, moet je niet vooraf al weten op welke stoel je zult zitten (early binding). Je kunt dat kiezen op het moment zelf (late binding).

Polymorfisme is goed voor onderhoudbaarheid, omdat we met polymorfisme gemakkelijker een bepaalde taak door een nieuw object kunnen laten uitvoeren. Het volstaat dat het nieuwe object een gelijkaardige interface heeft aan het bestaande object dat we willen vervangen.

2 De SOLID-principes

Het letterwoord SOLID staat voor vijf principes:

Single responsibility

Open/Closed

Liskov substitution

Interface segregation

Dependency inversion

Het zijn principes waarvan de ervaring geleerd heeft dat ze helpen om goede objecten, goede softwarebouwstenen te maken. Dat wil zeggen: objecten die geschikt zijn om er onderhoudsvriendelijke software mee te maken.

Deze principes zijn richtlijnen, geen wetten. Bij het toepassen is het zoeken naar een gulden middenweg. De principes niet toepassen is niet goed, maar ze te strikt toepassen is ook niet goed. Door ervaring leer je deze principes goed doseren.

Er zijn nog andere principes dan de vijf SOLID-principes, vaak ook met mooie letterwoorden als naam, om te je te helpen ze te onthouden. (Bijvoorbeeld: DRY: 'do not repeat yourself'. M.a.w.: schrijf nooit twee keer dezelfde code. KISS: 'keep it short and simple'. Of de GRASP-principes: 'General Responsibility Assignment Software Patterns' – zie daarvoor bijvoorbeeld (Larman, 2005)). Je ontmoet er beslist nog tijdens je loopbaan.

Veel van het materiaal uit dit hoofdstuk is ontleend aan (Fowler, Clean Code: a Handbook of Agile Software Craftsmanship, 2009), een uitstekend boek over het schrijven van onderhoudsvriendelijke software. Ook de andere werken van Martin Fowler zijn zeer aan te bevelen.

2.1 Single responsibility principle

Elke klasse moet één, en niet meer dan één reden hebben om gewijzigd te worden.

Dit principe heeft veel te maken met het principe van hoge cohesie dat onder andere in de cursus Analysis & Design aan bod kwam. Het principe van hoge cohesie zegt dat elke klasse één duidelijke, samenhangende verantwoordelijkheid moet hebben.

“Eén reden om gewijzigd te worden” wil zeggen: tijdens het verwachte onderhoud van het programma zal er maar één aanleiding zijn om de code van de klasse aan te passen.

Als dit principe gevolgd wordt is het gemakkelijker om te begrijpen waar een klasse voor dient dan als een klasse veel vage verantwoordelijkheden heeft.
Software waarvan je gemakkelijk begrijpt waar elke klasse voor dient is veel gemakkelijker te onderhouden dan software waar klassen onduidelijke verantwoordelijkheden hebben. Je kunt immers sneller de delen terugvinden die je moet aanpassen, en er is minder kans dat de aanpassing ongewenste neveneffecten veroorzaakt.

Wat zijn aanwijzingen dat dit principe toegepast wordt?

- klassen hebben een beperkte omvang
- de naam van de klasse maakt het doel van de klasse duidelijk
- je kunt de verantwoordelijkheid van de klasse omschrijven in een paar zinnen, liefst zonder gebruik van de woorden 'als', 'maar', 'en' en 'of'
- je kunt klassen gemakkelijk ordenen in groepen die bij elkaar horen

Let wel: dit zijn enkel aanwijzingen, geen garanties.

Bij klassen met te veel verantwoordelijkheden, of onduidelijke verantwoordelijkheden is het vaak zo dat

- de klasse erg groot is
- je zonder documentatie niet kunt begrijpen waar de klasse voor dient
- de klasse maar voor 1 specifieke toepassing bruikbaar is

Je kunt ook in de andere richting te ver gaan. Dan krijg je software met heel veel heel kleine klassen, waar je het overzicht in verliest.

Waarom helpt dit principe om onderhoudsvriendelijke code te krijgen?

Code die dit principe toepast maakt het gemakkelijker om terug te vinden welk stuk programma waarvoor verantwoordelijk is. Als je een programma moet aanpassen is het niet zo moeilijk om te vinden welke delen je juist moet aanpassen.

<u>Voorbeelden</u>

1. *Te veel redenen om te wijzigen*

```
public class SuperDashboard extends JFrame implements MetaDataUser {
  public Component getLastFocusedComponent();
  public void setLastFocused(Component lastFocused);
  public int getMajorVersionNumber();
  public int getMinorVersionNumber();
}
```

Deze klasse heeft twee redenen om te veranderen. Ten eerste doet ze iets met Java Swing componenten (ze is afgeleid van JFrame, een klasse die een venster voorstelt). Ten tweede doet ze iets met versie-informatie. Wanneer de componenten in het venster veranderen zal allicht ook het

versienummer veranderen, maar andersom is het perfect denkbaar dat het versienummer verandert zonder dat de componenten in het venster veranderen.

Het zou dus beter zijn om de verantwoordelijkheid voor de versie-informatie af te splitsen in een aparte klasse.

2. *Wordt single purpose toegepast in het factory method pattern?*

Factory method is een pattern dat een abstracte klasse voorziet die een object met een bepaalde interface creëert en gebruikt, en een concrete klasse, afgeleid van de abstracte klasse, die beslist welk concreet object geïnstantieerd zal worden.

De concrete klasse heeft 1 reden om gewijzigd te worden: je wilt iets wijzigen aan de creatie van het concrete object.
De abstracte klasse heeft 1 reden om gewijzigd te worden: je wilt iets wijzigen aan het gebruik van het object.

Het principe wordt hier dus toegepast.

3. *Wordt single purpose toegepast in het flyweight pattern?*

Flyweight is een pattern dat toegepast kan worden wanneer een programma een heleboel zware objecten, dus die veel geheugen nodig hebben, gebruikt. Zo een programma heeft at runtime veel geheugen nodig, en kan daardoor erg belastend zijn voor het systeem waar het op draait. Het pattern verkleint de memory footprint van het programma door een stuk van de toestand van objecten (de extrinsieke toestand) weg te halen uit die objecten, en die toestand door de client te laten beheren. De objecten hebben enkel nog hun intrinsieke toestand. Het gevolg daarvan is dat er veel minder instanties van die objecten nodig zijn, waardoor het programma minder geheugen nodig heeft.

De flyweight-klasse klasse heeft 1 reden om gewijzigd te worden: je wilt iets wijzigen aan de intrinsieke toestand van de flyweights.
De flyweight factory heeft 1 reden om gewijzigd te worden: je wilt iets wijzigen aan de creatie van de flyweights.
De client heeft meer dan 1 reden om gewijzigd te worden: de client wijzigt als er iets verandert aan de extrinsieke toestand van de flyweights, en ook als er iets verandert aan de andere verantwoordelijkheden van de client.

Het principe wordt hier dus maar gedeeltelijk toegepast.

2.2 Open/Closed principle

Een goede klasse is ‘open for extension’: ze kan uitgebreid worden.
Een goede klasse is ook ‘closed for modification’: haar gedrag moet niet gewijzigd worden.

Anders gezegd: we moeten software zo ontwerpen dat ze gemakkelijk aangepast kan worden door nieuwe code toe te voegen, zonder de bestaande code te veranderen.

Waarom mag het gedrag van een klasse niet gewijzigd worden? Een klasse wordt dikwijls op verschillende plaatsen gebruikt. Het zou kunnen dat als je het gedrag van een klasse wijzigt omdat dat voordelig is voor een bepaalde situatie, dat die wijziging ervoor zorgt dat de klasse niet meer goed werkt in de andere situaties!
Wanneer je een klasse aanpast, dan moet je alle software hertesten die die aangepaste klasse gebruikt. Dat kan veel meer zijn dan alleen het programma of het deel van het programma waar je op dat moment mee bezig bent.
Een voorbeeld: in een grote bedrijfstoepassing is er een klasse Product met een methode getPrice(). De returnwaarde van die methode is de prijs zonder BTW. Jij bent aan het werken aan een klein onderdeel van die toepassing. Je hebt daarvoor de prijs met BTW nodig. Je besluit getPrice() aan te passen zodat die methode voortaan de prijs met BTW retourneert. Je verandert dus het gedrag van de klasse Product. Op slag werken een heleboel andere delen van de toepassing die de klasse Product gebruiken niet meer correct.

Wat moet je dan wel doen als je een bestaande klasse wilt aanpassen aan je eigen noden? Je kunt ze uitbreiden, bijvoorbeeld door er een nieuwe klasse van af te leiden en in die nieuwe klasse de nodige aanpassingen te programmeren. In het geval van het voorbeeld: door een nieuwe klasse af te leiden van Product, en aan die nieuwe klasse een methode getPriceWithVAT() toe te voegen. De bestaande klasse blijft dan onveranderd, en alle programma's die die bestaande klasse gebruiken merken geen verandering en blijven dus werken.
Als je een klasse ontwerpt kun je ook voorzien dat bepaalde stukken aangepast zullen moeten worden. Door het gebruik van design patterns kun je aanpasbaarheid op een goede manier mogelijk maken. Het is wel niet gemakkelijk om vooraf te weten waar er aanpassing nodig zal zijn…

Wat zijn aanwijzingen dat dit principe toegepast wordt?

- functionaliteit wordt toegevoegd met nieuwe klassen, niet door het veranderen van bestaande klassen (dat zie je duidelijk in verschillende design patterns uit het handboek).
- er wordt overerving gebruikt
- er worden interfaces (in de Java-zin) gebruikt

Waarom helpt dit principe om onderhoudsvriendelijke code te krijgen?

Als je dit principe toepast, dan gebeurt onderhoud vooral door het toevoegen van nieuwe klassen, eerder dan het veranderen van bestaande klassen. Als bestaande klassen niet veranderen, dan blijft alle code die de diensten van die klassen gebruikt goed werken.

1. *Wordt open/closed toegepast in het factory method pattern?*

Factory method is een pattern dat een abstracte klasse voorziet die een object met een bepaalde interface creëert en gebruikt, en een concrete klasse, afgeleid van de abstracte klasse, die beslist welk concreet object geïnstantieerd zal worden.

Als de abstracte klasse een ander object nodig heeft, dan kan dat door een nieuwe concrete klasse af te leiden van de abstracte klasse. De abstracte klasse zelf moet niet gewijzigd worden, en ook niet de stukken van het programma die die abstracte klasse gebruiken. De abstracte klasse is dus open for extension en closed for modification.

Het principe wordt hier dus toegepast.

2. Wordt open/closed toegepast in het iterator pattern?

Iterator is een pattern dat toelaat om de elementen van een collectie een na een op te vragen, zonder dat je de concrete implementatie van de collectie moet kennen. Er zijn in dit pattern twee klassen betrokken: de collectie, en de iterator over de collectie.

De collectie kun je nieuwe functionaliteit geven door er een nieuwe klasse van af te leiden. De bestaande collectieklasse moet je niet veranderen. De concrete iteratorklasse die bij die collectie hoort moet je ook niet veranderen, op voorwaarde dat de interface van de collectie bruikbaar blijft voor de iterator.
De iterator kun je ook nieuwe functionaliteit geven door er een nieuwe klasse van af te leiden. Je moet dan wel de collectie aanpassen zodat de getIterator() van de collectie het nieuwe iteratortype retourneert.

Het principe wordt hier dus gedeeltelijk toegepast.

2.3 Liskov substitution principle

Het principe met de moeilijkste naam, dat echter het meest vanzelfsprekend lijkt.

Een object moet je altijd kunnen vervangen door een meer gespecialiseerd object, zonder dat dat invloed heeft op bestaande gebruikers van dat object.

Objectgeoriënteerde talen als Java of C# maken het gemakkelijk om dit principe te respecteren. Als klasse A afgeleid is van klasse B, dan kun je objecten van A overal gebruiken waar objecten van B bruikbaar zijn.

Maar zelfs in objectgeoriënteerde talen is het mogelijk om dit principe te schenden. Een klasse biedt een aantal diensten aan zijn gebruikers aan, heeft als het ware een contract met zijn gebruikers om bepaalde dingen te doen. Als een afgeleide klasse het contract verbreekt, dan is het Liskov-principe geschonden.
Bijvoorbeeld: in Java heeft klasse String een methode length(). Het ‘contract’ van klasse String met

zijn gebruikers is dat length() het aantal tekens in de string geeft. Je maakt nu een klasse MyString, afgeleid van String, met een override van length() die altijd -1 teruggeeft. MyString houdt zich dus niet aan het 'contract' van String, ook al kun je in principe in een programma een MyString gebruiken overal waar er een String staat. MyString schendt het Liskov subsitutieprincipe.

Hoe draagt dit principe bij aan onderhoudsvriendelijkheid?

Als je dit principe respecteert, dan kun je een bestaande klasse vervangen door een afgeleide klasse, zonder dat je je zorgen moet maken dat die vervanging bestaande code zal breken.

1. *Wordt het Liskov substitutieprincipe toegepast in het factory method pattern?*

Factory method is een pattern dat een abstracte klasse voorziet die een object met een bepaalde interface creëert en gebruikt, en een concrete factoryklasse, afgeleid van de abstracte klasse, die beslist welk concreet object geïnstantieerd zal worden.

Overal waar je de abstracte klasse bruikbaar is kun je die vervangen door een concrete factoryklasse die daarvan afgeleid is, als de concrete factory de afspraak respecteert dat die zorgt voor de creatie van een geschikt object. Dat is in dit pattern de bedoeling, dus het principe wordt toegepast.

2. *Wordt het Liskov substitutieprincipe toegepast in het decorator pattern?*

Decorator is een pattern dat toelaat om at runtime bijkomende functionaliteit te geven aan een object, door het in te pakken in een decorator-object. De decorator en het oorspronkelijke object zijn implementaties van eenzelfde interface, waardoor de client het ingepakte object kan gebruiken net zoals het oorspronkelijke object.

De client verwacht objecten die de interface implementeren. Voor de client kan een object dat de interface implementeert vervangen worden door een meer gespecialiseerd object dat ook de interface implementeert.
Als de decoratorklassen geschreven zijn in functie van de interface, en niet in functie van concrete types die ingepakt moeten worden, dan kunnen die concrete types vervangen worden door meer afgeleide types.
Het principe wordt hier dus toegepast.

2.4 Interface segregation principle

Een object mag niet afhankelijk zijn van methodes die het niet gebruikt.

Anders geformuleerd: een object mag enkel de methodes hebben die het echt nodig heeft.

Als je dit principe niet volgt, dan krijg je objecten met erg veel methodes. Die moeten allemaal correct werken, ook al gebruik je ze niet, want het zou kunnen dat jij of iemand anders die ooit toch eens zal gebruiken. Dergelijke objecten zijn dus lastiger te onderhouden dan objecten die het principe wel volgen.

Een manier om dit principe toe te passen die je dikwijls ziet zijn objecten die veel kleine interfaces (interfaces met weinig methodes) implementeren, eerder dan een paar heel grote. Door kleine interfaces te gebruiken heb je meer controle over welke methodes een object al of niet heeft.
In de Java-klassenbibliotheek zie je dit dikwijls toegepast worden, bijvoorbeeld bij de collectie-klassen.

1. *Wordt het interface segregationprincipe toegepast in het factory method pattern?*

Factory method is een pattern dat een abstracte klasse voorziet die een object met een bepaalde interface creëert en gebruikt, en een concrete factoryklasse, afgeleid van de abstracte klasse, die beslist welk concreet object geïnstantieerd zal worden.

Zowel de abstracte klasse als de concrete factoryklasse hebben enkel de methodes die ze nodig hebben. Het principe wordt dus toegepast.

2. *Wordt het interface segregationprincipe toegepast in het composite pattern?*

Composite is een pattern om boomstructuren te beheren. In composite zijn er twee soorten objecten: composite-objecten, dat zijn knopen in een boom die subknopen kunnen hebben, en leaf-objecten, dat zijn eindknopen. Beide zijn afgeleid van een gemeenschappelijke interface.

De gemeenschappelijke interface heeft methodes voor het beheer van subknopen en voor het gebruiken van de functionaliteit van knopen. De leaf-klasse implementeert de interface, en heeft dus methodes voor het beheer van subknopen. Maar de leaf-klasse kan geen subknopen hebben, en heeft die methodes dus niet nodig. De leafklasse is dus een schending van het interface segregationprincipe.

2.5 Dependency inversion principle

High-level modules mogen niet afhankelijk zijn van low-level modules, maar moeten afhankelijk zijn van abstracties.

Abstracties mogen niet van details afhankelijk zijn. Details moeten afhankelijk zijn van abstracties.

Een wat extreem voorbeeld om dit principe te verduidelijken: met Word kun je documenten opstellen en die opslaan. Om een document lokaal op een harde schijf op te slaan moeten er een heleboel bits op een bepaalde plaats op de schijf gezet worden.
Word is een high-level module. Het systeem dat de harde schijf stuurt is een low-level module. Voor

elk model harde schijf is het een beetje anders.
Het zou helemaal niet praktisch zijn als je een andere versie van Word nodig zou hebben voor elk model harde schijf dat er is. Daarom gebruikt Word een abstractie: het bestandssysteem. Het bestandssysteem is een abstractie van de bits die op een harde schijf staan. Op die manier heb je niet meer een versie van Word per model harde schijf, maar enkel nog een versie per soort bestandssysteem (dus per operating system).
Het bestandssysteem is een abstractie. Het zou niet praktisch zijn als het bestandssysteem (zoals gezien door de gebruikers van dat bestandssysteem) zou afhangen van het type harde schijf dat in een computer zit – het model harde schijf is een detail. Dat zou willen zeggen dat als je de harde schijf van een computer vervangt, je ook een nieuw bestandssysteem zou moeten installeren.

Alle software van enige omvang heeft high-level modules, zoals de user interface of de controller, en low-level modules, zoals data access objecten, of timers, of sockets. Als de high-level modules afhankelijk zijn van die low-level modules dan wil dat zeggen dat je die low-level modules niet zomaar kunt veranderen of vervangen. Dat maakt de software moeilijker te onderhouden, want een verandering in een low-level module (een detail) kan dan de werking van de high-level modules (het belangrijke) verstoren.

Dit principe is verwant met het principe van lage koppeling. Een high-level module die van low-level modules afhankelijk is heeft een hoge koppeling.

1. *Wordt het dependency inversionprincipe toegepast in het factory method pattern?*

Factory method is een pattern dat een abstracte klasse voorziet die een object met een bepaalde interface creëert en gebruikt, en een concrete factoryklasse, afgeleid van de abstracte klasse, die beslist welk concreet object geïnstantieerd zal worden.

De hoofdzaak in het pattern is de functionaliteit van de abstracte klasse, dus wat er gebeurt met het concrete object dat de concrete factory maakt. Het concrete type van het concrete object is een implementatiedetail, en is minder belangrijk.
De concrete factoryklasse is afgeleid van de abstracte klasse. Dat maakt de concrete factoryklasse afhankelijk van de abstracte klasse. Andersom is de abstracte klasse niet afhankelijk van de concrete factory. De hoofdzaak is dus niet afhankelijk van het detail.
De client gebruikt de abstracte klasse, niet de concrete factory. De client gebruikt dus een abstractie, niet iets concreet.

Het principe wordt dus toegepast in factory method.

2. *Wordt het dependency inversionprincipe toegepast in het memento pattern?*

Memento is een pattern dat toelaat om de toestand van een object (de originator) op te slaan, zodat je het object indien gewenst later naar die toestand kunt terugbrengen.

Bij het pattern is de originatorklasse afhankelijk van de mementoklasse: de originator kent zijn memento, maar de memento kent zijn originator niet. Maar de originator is de hoofdzaak, en de memento het detail. Dit ontwerp respecteert het dependency inversionprincipe dus niet.

Bibliografie

Fowler, M. (2003). *Patterns of Enterprise Application Architecture.* Addison-Wesley.

Fowler, M. (2009). *Clean Code: a Handbook of Agile Software Craftsmanship.* Prentice Hall.

Larman, C. (2005). *Applying UML and Patterns.* Pearson.

Martin, R. C. (n.d.). *The Clean Code Blog*. Retrieved from http://blog.cleancoder.com